

PROJECT REPORT

SPENDWISE

Personal Expense Tracker

PROBLEM CHOSEN:

The problem I decided to work on is something that I think is very common, especially among college students. It is about managing personal expenses and keeping track of where our money is actually going. I chose this problem because I have noticed that it is very easy to spend small amounts of money throughout the day without really keeping track of them.

As students, we have many small expenses. It can be food from the mess or canteen, snacks, drinks, travelling, shopping, stationery or other things. Most of the time, we remember the bigger expenses but forget about the smaller ones. For example, spending ₹20 or ₹50 may not feel like a big deal at that moment, but when many such expenses happen during the week, the total amount can become quite large.

I also feel that keeping everything in our memory is not a very reliable way of managing money. Sometimes we know that we spent money somewhere, but later we don't remember exactly how much we spent or what we spent it on. This makes it difficult to understand our spending habits.

Because of this, I decided to make a simple Personal Expense Tracker using Java. I wanted to make something that is easy to use and can help a person record their expenses instead of trying to remember everything.

WHY THIS PROBLEM MATTERS?

Initially, tracking expenses may not seem like a very important problem. If someone spends ₹20 or ₹30 on something, it usually does not feel like a big expense. However,

the problem starts when these small expenses keep happening regularly and we don't notice the total amount.

For a student, money is usually limited and there are many things that need to be paid for. Because of this, knowing where the money is being spent can actually be useful. If we record our expenses, we can look at them later and understand which categories are taking up most of our money.

Another reason this problem matters is that people often don't have a proper record of their daily spending. They may know their balance but not know how they reached that balance. A simple expense tracker can make this information easier to understand.

I did not want to choose a problem that was only theoretical. I wanted to work on something that can actually be used in daily life. Expense tracking is simple, but it is also practical and relatable to students.

MY APPROACH TO SOLVING THE PROBLEM:

I first tried to think about what the most basic things an expense tracker should be able to do. I did not want to make the project unnecessarily complicated, so I started with the main functions that a person would actually need while keeping track of expenses.

I decided that the user should be able to add an expense by entering details such as an ID, amount, category, description and date. After adding an expense, the user should also be able to see all the expenses that have been recorded.

After that, I added options to search expenses by category and calculate the total amount spent. I also added a delete option so that an expense can be removed if it was entered incorrectly or is no longer needed.

One problem I thought about was that the expenses should not disappear every time the program is closed. To solve this, I used file handling and stored the expense details in a text file. When the program starts, the saved expenses can be loaded again.

I divided the program into different classes instead of putting everything in one file. I used Expense for storing the details of an expense, ExpenseManager for managing the expenses and Main for the menu and user interaction.

KEY DECISIONS I MADE:

One of my main decisions was to keep the project as a command-line application. I decided not to use a graphical interface because the main purpose of the project was to practice Java concepts and create a working application that could be run directly from the terminal.

I decided to use ArrayList for storing the expenses because the number of expenses can change while the program is running. It also made it easier to add and remove expense objects.

I also decided to use separate classes for different responsibilities. This made the code easier to understand and allowed me to keep the expense details separate from the operations performed on them.

For storing the data, I chose a simple text file instead of using a database. Since this is a small project, I felt that file handling was enough for what I wanted the application to do.

Another decision I made was to give every expense an ID. This makes it easier to identify a particular expense, especially when deleting it. I also added a check so that the same ID cannot be added twice.

CHALLENGES I FACED:

One of the difficulties that I encountered was learning how to save and retrieve expense information from the file. It was easy to store the data during the execution of the program; however, I wanted this data to still be there even after closing the program. I needed to learn how files are written to and read in Java.

A difficult aspect of the program is how to deal with different operations. The operation for adding an expense was easy; however, the operations for viewing, searching, and deleting required me to go through the ArrayList and check the information.

There is also the problem of what will happen if the user deletes the non-existing ID. Rather than having the program crash, I added an option where the program returns a message stating that there is no such expense.

Lastly, a simple problem that I have to consider was duplicate IDs. Having duplicate IDs in the expense list can be problematic when the user tries to delete one of them. To prevent this, I added an additional step before creating an expense.

WHAT I LEARNED:

While making this project, I learned how different Java concepts can be combined to make an actual working application instead of using them only in separate programs.

I got more practice with classes and objects because each expense is represented as an object. I also understood better how an ArrayList can be used to store and manage multiple objects.

I learned more about file handling and how information can be written into a file and read again when the program starts. This was one of the more useful parts for me because it showed me how a program can keep data even after it has been closed.

I also learned that small things like checking duplicate IDs and handling errors are important in a project. A program should not only work when everything is entered correctly; it should also handle simple mistakes without crashing.

Most importantly, I learned how to break a problem into smaller parts and then implement each part separately. This made the project easier to understand and also made the code more organized.

CONCLUSION:

Making this Personal Expense Tracker helped me understand how programming can be used to solve a simple problem that people face in everyday life. I started with the basic idea of recording expenses and then added the functions needed to manage those records.

The final project allows the user to add, view, search and delete expenses, calculate the total expenses, and save the records for later use. I also added checks for duplicate IDs and handled errors while working with the stored data.

I think, there's a lot that can be improved. It's very basic right now, but it could be improved if I added more features or made it more interactive. I suppose, though, for right now, it does serve its purpose.

The project is not very complicated, but that was intentional because I wanted to understand each part properly instead of adding features that I could not explain.

Overall, this project gave me better practice with Java and helped me understand how concepts like classes, objects, ArrayList, methods and file handling can work together in one complete project.

STUDENT NAME: KRISHNA SINGH

REGISTRATION NUMBER: 25BAI10124